

# Hybrid XOR–Rotation–SHA256 Algorithm.

---



**Section: 5w6**

**Supervised By: Dr. Dalal Ali.Adlaim**



## Course\_ Information Security

**“IT311”**

| Serial | Section | NAME                       | ID        | Email                |
|--------|---------|----------------------------|-----------|----------------------|
|        |         |                            |           |                      |
| 1      | 5W6     | Fajr Omar Alrubeaan        | 445008461 | 445008461@pnu.edu.sa |
| 2      | 5W6     | Shujon Ali ALShehri        | 445008413 | 445008413@pnu.edu.sa |
| 3      | 5W6     | Nora Sultan ALAnzi         | 445008318 | 445008318@pnu.edu.sa |
| 4      | 5W6     | Layan Abdullah<br>ALmathum | 445008523 | 445008523@pnu.edu.sa |

**First Semester 2025**



## Table of Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| ➤ INTRODUCTION: .....                                       | 3  |
| ➤ ENCRYPTION: .....                                         | 3  |
| ➤ DECRYPTION: .....                                         | 3  |
| ➤ KEY: .....                                                | 4  |
| ➤ BLOCK DIAGRAM ENCRYPTION: .....                           | 4  |
| ➤ FLOW CHARTS: .....                                        | 5  |
| ➤ SCREENSHOTS OF THE CODE METHODS: .....                    | 6  |
| ➤ SCREENSHOTS OF THE OUTPUTS ENCRYPTION: .....              | 8  |
| ➤ SCREENSHOTS OF THE OUTPUTS DECRYPTION: .....              | 9  |
| ➤ Fail Cases of Integrity Verification Demonstration: ..... | 10 |
| ➤ REFERENCE: .....                                          | 11 |
| ➤ APPENDIX: .....                                           | 12 |

## ➤ INTRODUCTION:

This project uses a hybrid encryption method that combines XOR encryption, character rotation, and SHA-256 hashing to protect data. First, the plaintext is encrypted using XOR with a numeric key and then rotated to produce the final ciphertext. Next, a SHA-256 hash is generated to ensure that the data has not been altered. During decryption, the rotation and XOR steps are reversed to recover the original text, and the hash is recalculated and compared with the original value. If both hashes match, the integrity of the data is verified. This hybrid method provides both confidentiality and integrity in a simple and effective way.

## ➤ ENCRYPTION:

1. Enter the original text (plaintext) that the user wants to encrypt.
2. Enter the XOR key (number) to apply the XOR operation to each character of the text.
3. Enter the rotation value to rotate the characters after the XOR operation.
4. Apply the XOR operation on the original text using the key.
5. Apply the rotation operation on the XOR result to get the final ciphertext.
6. Generate the SHA-256 hash of the ciphertext to verify data integrity later (optional).
7. Display the results:
  - Original text (Plaintext).
  - XOR key and rotation value.
  - Ciphertext (Encrypted text).
  - SHA-256 hash of the ciphertext.

## ➤ DECRYPTION:

1. Enter the ciphertext that the user wants to decrypt.
2. Enter the XOR key used during encryption.
3. Enter the rotation value used during encryption.
4. Reverse the rotation (subtract the rotation value from each character) to restore the text after un-rotation.
5. Reverse the XOR operation on the text after un-rotation using the same key.
6. The text returns to its original state (plaintext) after decryption.
7. Verify the integrity of the text using the original SHA-256 hash (optional).
8. Display the results:
  - Decrypted text.
  - SHA-256 hash of the text after decryption.
  - Integrity verification result if the original hash is provided.

### ➤ KEY:

Our algorithm uses a symmetric numeric key for XOR encryption and a rotation value for the second layer.

Both encryption and decryption use the same key and rotation value, since XOR and rotation are reversible operations.

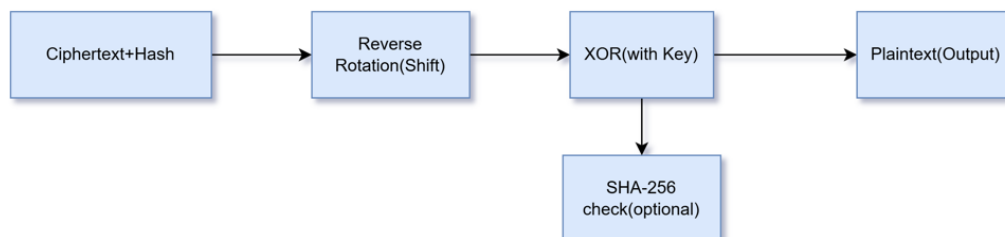
Our structure uses one shared logic for both, simplifying the program while maintaining correctness.

This design still provides confidentiality (through XOR and rotation) and integrity (through SHA-256 hashing).

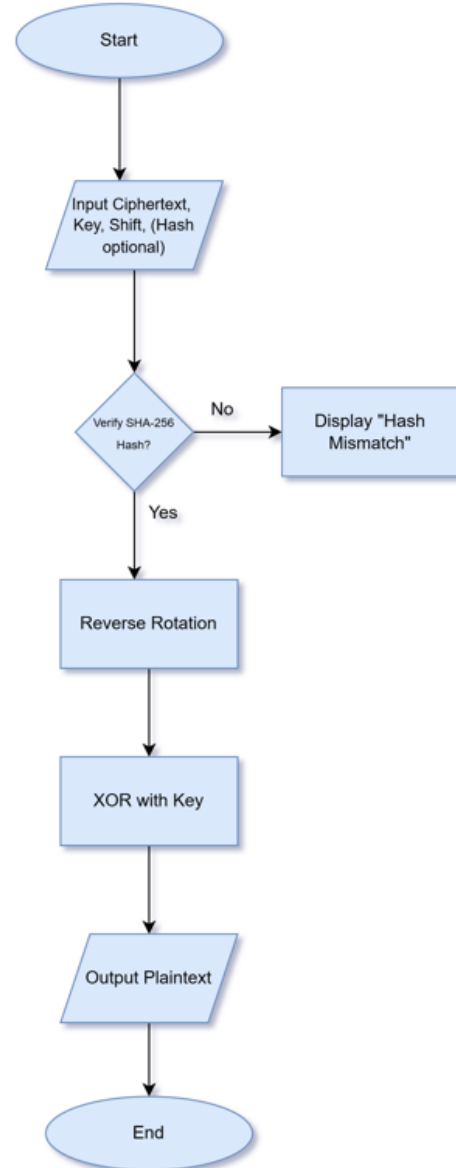
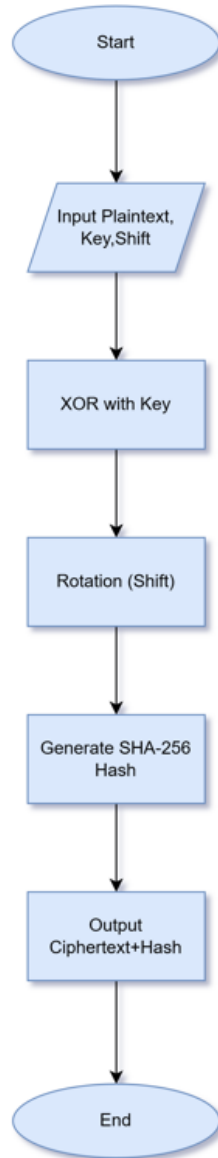
### ➤ BLOCK DIAGRAM ENCRYPTION:



### ➤ BLOCK DIAGRAM DECRYPTION:



➤ **FLOW CHARTS:**



➤ **SCREENSHOTS OF THE CODE METHODS:**

```
public class HybridXORRotationHash {  
  
    // XOR encryption/decryption  
    static String xorEncryptDecrypt(String text, int key) {  
        StringBuilder result = new StringBuilder();  
        for (int i = 0; i < text.length(); i++) {  
            result.append((char) (text.charAt(index:i) ^ key));  
        }  
        return result.toString();  
    }  
}
```

```
// Rotation encryption/decryption  
static String rotate(String text, int shift) {  
    StringBuilder result = new StringBuilder();  
    for (int i = 0; i < text.length(); i++) {  
        result.append((char) (text.charAt(index:i) + shift));  
    }  
    return result.toString();  
}
```

- **Hashing steps:**

- 1- After encryption, the ciphertext is converted into bytes.
- 2- These bytes are processed by SHA-256 to produce a fixed hash value.
- 3- The hash is saved so it can be used later to check if the data stayed the same.

```
// SHA-256 hash function
static String sha256(String text) throws Exception {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] digest = md.digest(input: text.getBytes());
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) {
        hex.append(str: String.format(format: "%02x", args: b));
    }
    return hex.toString();
}
```

---

- **Hash verification steps:**

- 1- During decryption, a new hash is generated from the ciphertext.
- 2- The new hash is compared with the original one.
- 3- If they match, the data is intact; if not, it has been altered.

```
// Integrity check: compares recalculated SHA-256 hash with the original to confirm authenticity.
if (!hashOriginal.isEmpty()) {
    boolean verified = hashCipherAgain.equals(anObject: hashOriginal);
    System.out.println("Integrity Verified: " + verified);
} else {
    System.out.println(x: "No hash provided — integrity check skipped.");
}
```

---

## ➤ SCREENSHOTS OF THE OUTPUTS ENCRYPTION:

```
=== Hybrid XOR-Rotation-SHA256 Algorithm ===  
  
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit  
Enter your choice: 1  
Enter your name (plaintext): Fajr  
Enter XOR key: 4  
Enter rotation value: 3  
  
--- ENCRYPTION RESULT ---  
Plaintext: Fajr  
Key: 4 | Rotation: 3  
Ciphertext: Ehqy  
SHA-256 of Ciphertext: 435de611e55a714fala0e664dd6edbf0793a58df893clcc0c48c57cle7acd6l  
Save your Ciphertext and Hash for later decryption.
```

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit  
Enter your choice: 1  
Enter your name (plaintext): Layan  
Enter XOR key: 5  
Enter rotation value: 3  
  
--- ENCRYPTION RESULT ---  
Plaintext: Layan  
Key: 5 | Rotation: 3  
Ciphertext: Lg gn  
SHA-256 of Ciphertext: 59d429e95a0lda02deaf21e8e075fc07ca6ab46fdfa15d6457f547dc88f8d4ed  
Save your Ciphertext and Hash for later decryption.
```

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit  
Enter your choice: 1  
Enter your name (plaintext): Nora  
Enter XOR key: 6  
Enter rotation value: 3  
  
--- ENCRYPTION RESULT ---  
Plaintext: Nora  
Key: 6 | Rotation: 3  
Ciphertext: Klwj  
SHA-256 of Ciphertext: 9f9dce31b1089130e4785820eb749c437bad61b0b26495a0f559a8855374ff40  
Save your Ciphertext and Hash for later decryption.
```

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit  
Enter your choice: 1  
Enter your name (plaintext): Shojun  
Enter XOR key: 7  
Enter rotation value: 3  
  
--- ENCRYPTION RESULT ---  
Plaintext: Shojun  
Key: 7 | Rotation: 3  
Ciphertext: Wrkpul  
SHA-256 of Ciphertext: a4d0f9c49a6a4dacf04ae719301833a93beac6b846fddb1ade18dec8a26fcee4  
Save your Ciphertext and Hash for later decryption.
```



## ➤ SCREENSHOTS OF THE OUTPUTS DECRYPTION:

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit
Enter your choice: 2
Enter Ciphertext to decrypt: Ehqy
Enter XOR key used for encryption: 4
Enter rotation value used for encryption: 3
Enter the original SHA-256 hash (optional, press Enter to skip): 435de611e55a714fala0e664dd6edbf0793a58df893c1cc0c48c57clee7acd61

--- DECRYPTION RESULT ---
Decrypted Text: Fajr
Recalculated SHA-256 of Ciphertextt: 435de611e55a714fala0e664dd6edbf0793a58df893c1cc0c48c57clee7acd61
Integrity Verified: true
```

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit
Enter your choice: 2
Enter Ciphertext to decrypt: Lg gn
Enter XOR key used for encryption: 5
Enter rotation value used for encryption: 3
Enter the original SHA-256 hash (optional, press Enter to skip): 59d429e95a01da02deaf21e8e075fc07ca6ab46fdfa15d6457f547dc88f8d4ed

--- DECRYPTION RESULT ---
Decrypted Text: Layan
Recalculated SHA-256 of Ciphertextt: 59d429e95a01da02deaf21e8e075fc07ca6ab46fdfa15d6457f547dc88f8d4ed
Integrity Verified: true
```

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit
Enter your choice: 2
Enter Ciphertext to decrypt: Klwj
Enter XOR key used for encryption: 6
Enter rotation value used for encryption: 3
Enter the original SHA-256 hash (optional, press Enter to skip): 9f9dce31b1089130e4785820eb749c437bad61b0b26495a0f559a8855374ff40

--- DECRYPTION RESULT ---
Decrypted Text: Nora
Recalculated SHA-256 of Ciphertextt: 9f9dce31b1089130e4785820eb749c437bad61b0b26495a0f559a8855374ff40
Integrity Verified: true
```

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit
Enter your choice: 2
Enter Ciphertext to decrypt: Wrkpul
Enter XOR key used for encryption: 7
Enter rotation value used for encryption: 3
Enter the original SHA-256 hash (optional, press Enter to skip): a4d0f9c49a6a4dacf04ae719301833a93beac6b846fddb1ad18dec8a26fcee4

--- DECRYPTION RESULT ---
Decrypted Text: Shojun
Recalculated SHA-256 of Ciphertextt: a4d0f9c49a6a4dacf04ae719301833a93beac6b846fddb1ad18dec8a26fcee4
Integrity Verified: true
```

## ➤ Fail Cases of Integrity Verification Demonstration:

### 1. In case No hash entered → integrity check skipped.

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit
Enter your choice: 1
Enter your name (plaintext): Taif
Enter XOR key: 2
Enter rotation value: 3

--- ENCRYPTION RESULT ---
Plaintext: Taif
Key: 2 | Rotation: 3
Ciphertext: Yfng
SHA-256 of Ciphertext: e94b5fa6d59810a749e46d4bda6cd50c3b3e5799e724e8bb566eee898a9641bc
Save your Ciphertext and Hash for later decryption.

Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit
Enter your choice: 2
Enter Ciphertext to decrypt: Yfng
Enter XOR key used for encryption: 2
Enter rotation value used for encryption: 3
Enter the original SHA-256 hash (optional, press Enter to skip):

--- DECRYPTION RESULT ---
Decrypted Text: Taif
Recalculated SHA-256 of Ciphertext: e94b5fa6d59810a749e46d4bda6cd50c3b3e5799e724e8bb566eee898a9641bc
No hash provided — integrity check skipped.
```

### 2. In case Wrong ciphertext:

An extra space was entered in the ciphertext, which changed the hash and caused the integrity verification to fail. → integrity check (false).

```
Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit
Enter your choice: 1
Enter your name (plaintext): Taif
Enter XOR key: 2
Enter rotation value: 3

--- ENCRYPTION RESULT ---
Plaintext: Taif
Key: 2 | Rotation: 3
Ciphertext: Yfng
SHA-256 of Ciphertext: e94b5fa6d59810a749e46d4bda6cd50c3b3e5799e724e8bb566eee898a9641bc
Save your Ciphertext and Hash for later decryption.

Choose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit
Enter your choice: 2
Enter Ciphertext to decrypt: Yfng
Enter XOR key used for encryption: 2
Enter rotation value used for encryption: 3
Enter the original SHA-256 hash (optional, press Enter to skip): e94b5fa6d59810a749e46d4bda6cd50c3b3e5799e724e8bb566eee898a9641bc

--- DECRYPTION RESULT ---
Decrypted Text: Taif
Recalculated SHA-256 of Ciphertext: a0130225d9280ca4a834c90649060a337502e6dc6c59f97e7fdc9b0c478dea30
Integrity Verified: false
```



➤ **REFERENCE:**

- Oracle. (2025). MessageDigest (Java SE 11 & JDK 11). Oracle Help Center.  
<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/security/MessageDigest.html>
- Oracle. (2025). StringBuilder (Java SE 11 & JDK 11). Oracle Help Center.  
<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/StringBuilder.html>
- Oracle. (2025). StandardCharsets (Java SE 11 & JDK 11). Oracle Help Center.  
<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/nio/charset/StandardCharsets.html>

➤ **APPENDIX:**

```
import java.security.MessageDigest;
import java.util.Scanner;

public class HybridXORRotationHash {

    // XOR encryption/decryption
    static String xorEncryptDecrypt(String text, int key) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < text.length(); i++) {
            result.append((char) (text.charAt(i) ^ key));
        }
        return result.toString();
    }

    // Rotation encryption/decryption
    static String rotate(String text, int shift) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < text.length(); i++) {
            result.append((char) (text.charAt(i) + shift));
        }
        return result.toString();
    }

    // SHA-256 hash function
    static String sha256(String text) throws Exception {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] digest = md.digest(text.getBytes());
        StringBuilder hex = new StringBuilder();
        for (byte b : digest) {
            hex.append(String.format("%02x", b));
        }
        return hex.toString();
    }
}
```

```
public static void main(String[] args) throws Exception {
    Scanner sc = new Scanner(System.in);
    System.out.println("=== Hybrid XOR–Rotation–SHA256 Algorithm ===");
    boolean running = true;

    while (running) {
        System.out.println("\nChoose Mode: 1 = Encrypt | 2 = Decrypt | 3 = Exit");
        System.out.print("Enter your choice: ");
        int mode = sc.nextInt();
        sc.nextLine(); // consume newline

        if (mode == 1) {
            // === ENCRYPTION MODE ===
            System.out.print("Enter your name (plaintext): ");
            String name = sc.nextLine();

            System.out.print("Enter XOR key: ");
            int key = sc.nextInt();

            System.out.print("Enter rotation value: ");
            int shift = sc.nextInt();
            sc.nextLine();

            String xorCipher = xorEncryptDecrypt(name, key);
            String rotatedCipher = rotate(xorCipher, shift);
            String hashCipher = sha256(rotatedCipher);

            System.out.println("\n--- ENCRYPTION RESULT ---");
            System.out.println("Plaintext: " + name);
            System.out.println("Key: " + key + " | Rotation: " + shift);
            System.out.println("Ciphertext: " + rotatedCipher);
            System.out.println("SHA-256 of Ciphertext: " + hashCipher);
            System.out.println("Save your Ciphertext and Hash for later decryption.");
        }

        else if (mode == 2) {
            // === DECRYPTION MODE ===
            System.out.print("Enter Ciphertext to decrypt: ");
            String cipher = sc.nextLine();
        }
    }
}
```



```
System.out.print("Enter XOR key used for encryption: ");
int key = sc.nextInt();

System.out.print("Enter rotation value used for encryption: ");
int shift = sc.nextInt();
sc.nextLine();

System.out.print("Enter the original SHA-256 hash (optional, press Enter to skip): ");
String hashOriginal = sc.nextLine().trim();

String unrotated = rotate(cipher, -shift);
String decrypted = xorEncryptDecrypt(unrotated, key);
String hashCipherAgain = sha256(cipher);

System.out.println("\n--- DECRYPTION RESULT ---");
System.out.println("Decrypted Text: " + decrypted);
System.out.println("Recalculated SHA-256 of Ciphertextt: " + hashCipherAgain);

// Integrity check: compares recalculated SHA-256 hash with the original to confirm
authenticity.
if (!hashOriginal.isEmpty()) {
    boolean verified = hashCipherAgain.equals(hashOriginal);
    System.out.println("Integrity Verified: " + verified );
} else {
    System.out.println("No hash provided — integrity check skipped.");
}
}

else if (mode == 3) {
    System.out.println("\nExiting program... Goodbye ");
    running = false;
}

else {
    System.out.println("Invalid choice! Please enter 1, 2, or 3.");
}
}
}
}
```